Documentación Completa del Proyecto — Restaurant Nómina Xpress

> Documento técnico para desarrolladores y mantenedores. Para el uso funcional del sistema por parte del cliente, ver MANUAL_DE_USUARIO.md. Última revisión de contenido contra el código: **2026-07-31** (commit `67ce86a`). Autenticación, seguridad, entorno, migraciones y mantenimiento revisados el **2026-11-05** (commit `319db0c`, Fase 0 del Plan de Calidad v3.2).

Tabla de contenido

1. Introducción general
2. Objetivo del proyecto
3. Alcance
4. Visión general del sistema
5. Arquitectura
6. Estructura de carpetas y archivos
7. Tecnologías y su propósito
8. Dependencias principales
9. Configuración del entorno
10. Instalación
11. Ejecución local
12. Base de datos
13. Módulos principales
14. Funcionalidades, una por una
15. Roles, permisos y restricciones
16. Flujos de trabajo principales
17. Reglas de negocio
18. Validaciones importantes
19. Manejo de errores
20. Seguridad
21. Auditabilidad y trazabilidad
22. Despliegue y producción
23. Mantenimiento
24. Recomendaciones para futuros desarrolladores
25. Limitaciones actuales
26. Posibles mejoras futuras

1. Introducción general

Restaurant Nómina Xpress es una aplicación web para la gestión de nómina de restaurantes que se **despliega de forma independiente para cada cliente**: cada cliente tiene su **propio despliegue y su propia base de datos** (un único restaurante por base). El administrador registra las horas trabajadas y el sistema calcula automáticamente los salarios aplicando **tarifas diferenciadas para domingos y festivos colombianos**, gestiona **propinas, bonos, descuentos y ajustes de pago**, y genera **reportes quincenales y mensuales en PDF y Excel**.

> **Modelo de despliegue (importante):** el sistema **no** opera como un SaaS multi-tenant compartido (muchos clientes en una misma instancia/BD). Cada cliente se despliega por separado con su propia base de datos. El esquema conserva una columna `tenantId` en cada tabla como aislamiento de datos (herencia del diseño), pero en la práctica cada base de datos contiene **un solo tenant**.

2. Objetivo del proyecto

Automatizar y hacer trazable el cálculo de nómina quincenal de un restaurante, eliminando el cálculo manual propenso a errores, con soporte para:

- Tarifa normal vs. tarifa especial (domingos/festivos) por empleado.
- Turnos partidos (dos bloques de entrada/salida por día).
- Propinas repartidas de forma proporcional a las horas efectivas.
- Bonos y descuentos configurables (fijos o por empleado, quincenales o mensuales).
- Control de acceso granular por rol, rol personalizado y permiso individual.
- Auditoría inmutable de todas las acciones sensibles.

3. Alcance

Incluye: autenticación por credenciales, gestión de personal, registro de horas, horarios/turnos, propinas, bonos, descuentos, ajustes de pago, cálculo de nómina, reportes PDF/Excel, portal del empleado, roles y permisos, auditoría, notificaciones push, personalización por tenant (colores/logo), backups y migraciones por cliente.

No incluye (fuera de alcance): el **módulo de inventario** es una aplicación **externa** e independiente; Nómina Xpress solo actúa como **fuente de verdad de permisos** para él (ver docs/INTEGRACION_INVENTARIO.md). La integración está **desactivada por defecto** mediante feature flag. Tampoco se incluye facturación electrónica, contabilidad ni liquidación de prestaciones sociales/seguridad social.

4. Visión general del sistema

- **Dos portales:** el **área de administración** (`/admin/*`) para gestión y nómina, y el **portal del empleado** (`/portal/*`) para consultar su quincena y su horario.
- **Despliegue independiente por cliente:** cada cliente corre en su propio despliegue, aislado del resto (no comparte instancia ni base de datos con otros clientes).
- **Un solo código base** (`main`) alimenta todos los despliegues; la diferencia entre clientes se configura **solo con variables de entorno** en Vercel.
- **Base de datos por cliente:** cada cliente tiene su propia base **Turso (libSQL)** en producción (un único tenant por base); en local se usa un archivo SQLite (`file:./dev.db`) con el mismo adapter.

5. Arquitectura

5.1 Patrón general

Aplicación **Next.js 16 (App Router)** con renderizado del lado del servidor:

- **Server Components** leen de la base de datos **directamente** vía el cliente Prisma (`src/lib/db.ts`). Nunca hacen `fetch` a una API route propia.
- **Mutaciones** (crear/editar/borrar) van por **API routes** (`POST/PUT/DELETE` en `src/app/api/*`) invocadas desde componentes cliente.
- **Autorización fina** en cada página/API mediante permisos efectivos resueltos contra la BD; el middleware (`src/proxy.ts`) solo hace el gating grueso por portal.

5.2 Capa de datos (Prisma 7 + libSQL)

Prisma 7 usa un motor "client" que requiere un **adapter**. El proyecto usa `@prisma/adapter-libsql` con `@libsql/client` para local y producción por igual (`src/lib/db.ts`):

```
const adapter = new PrismaLibSql({
  url: process.env.TURSO_DATABASE_URL!,        // file:./dev.db en local; libsql://… en prod
  authToken: process.env.TURSO_AUTH_TOKEN,
});
export const prisma = new PrismaClient({ adapter });
```

- El `schema.prisma` **no** define `url` en el `datasource` — la URL vive en `prisma.config.ts` (que lee `TURSO_DATABASE_URL ?? "file:./dev.db"`).
- El cliente Prisma se genera en `src/generated/prisma/` (output custom).
- En desarrollo, la instancia de Prisma se cachea en `globalThis` para evitar múltiples conexiones con el hot-reload.

5.3 Autenticación (NextAuth v5)

- Proveedor **Credentials** (usuario + contraseña; contraseñas con `bcrypt`).
- Estrategia de sesión **JWT**. El token transporta: `id` , `role` , `tenantId` , `employeeId` , `inventoryAccess` , `inventoryPermissions` y `sv` (versión de sesión).
- **Duración:** 12 h sin actividad, renovada cada hora (antes, 30 días por defecto).
- **Revocación (RN-19):** cada `User` tiene `sessionVersion` . En cada request el callback `jwt()` compara la versión del token con la de la BD y devuelve `null` (sesión cerrada, cookie borrada) si el usuario fue eliminado, desactivado o su versión cambió. Se incrementa al desactivar, al cambiar contraseña, usuario, rol o rol personalizado (`sessionRevocationData()` de `src/lib/session-guard.ts`). Un token sin `sv` (emitido antes del 2026-11-05) cuenta como versión 0.
- Un usuario desactivado no tiene permisos efectivos aunque su token siga vivo.
- `src/lib/auth.ts` exporta `handlers` , `auth` , `signIn` , `signOut` .
- El login/logout/intentos fallidos quedan registrados en auditoría.

- Los **permisos de inventario se refrescan en cada request** (no se congelan en el JWT de login), para que el toggle de acceso y la matriz granular queden sincronizados.

Cookie de sesión compartida entre subdominios

Para el modo integrado con Inventory Xpress, la variable `AUTH_COOKIE_DOMAIN` (p. ej. `.cucinadeifiori.com`) hace que la cookie de sesión se emita con ese `Domain`, de modo que el subdominio del inventario (`inventario.<dominio>`) la lea y valide el JWT con el **mismo** `NEXTAUTH_SECRET`.

- Cuando está definida, la cookie se llama `__Secure-nx.session-token` (nombre propio a propósito, no el default `__Secure-authjs.session-token`): así la app **ignora** cookies host-only viejas que quedaron en los navegadores durante la transición, y el login funciona sin pedir al usuario que borre cookies.
- El nombre **debe ser idéntico en nómina e inventario** — forma parte de la sal de cifrado del JWT.
- Si **no** está definida (demo y local), NextAuth usa su cookie host-only por defecto y nada cambia.

5.4 Middleware (`src/proxy.ts`)

En Next.js 16 el middleware se llama `proxy.ts` (export default, compatible con el wrapper `auth()` de NextAuth). Corre en **edge runtime**, por lo que **no** puede resolver permisos desde la BD; solo:

- Redirige a `/login` si no hay sesión.
- Envía a los `EMPLOYEE` al `/portal`, y al resto al `/admin/dashboard`.
- El control fino se hace en `admin/layout` (Server Component con BD fresca).

6. Estructura de carpetas y archivos

```
restaurant-nomina/
├── prisma/
│   ├── schema.prisma              # Modelo de datos (19 modelos)
│   ├── migrations/                # Migraciones SQL versionadas
│   ├── seed.ts                    # Seed mínimo (npm run db:seed)
│   ├── seed-local.ts              # Seed integral de demostración
│   ├── run-migrations.mjs         # Aplica migraciones a Turso (usado por CI)
│   └── turso-*.mjs                # Scripts puntuales de migración a Turso
├── src/
│   ├── app/
│   │   ├── (auth)/login/          # Login (único route group)
│   │   ├── admin/                 # Área de administración
│   │   │   ├── dashboard/  employees/  time-entries/  schedules/
│   │   │   ├── tips/  reports/  roles/  usuarios/  audit/  settings/
│   │   │   ├── holidays/           # Festivos personalizados del cliente
│   │   │   ├── profile/  my-quincena/  my-horario/
│   │   ├── portal/                # Portal del empleado
│   │   │   ├── report/  schedule/  profile/
│   │   ├── api/                   # API routes
│   │   │   ├── admin/*  employee/*  auth/*  cron/*  inventory-permissions/*
│   │   ├── layout.tsx  page.tsx  globals.css
│   ├── components/
│   │   ├── ui/                    # Primitivas shadcn/ui
│   │   ├── admin/                 # Componentes del admin (forms, managers, clients)
│   │   ├── portal/                # Componentes del portal
│   │   └── shared/                # AdminSidebar, PortalNav, ThemeProvider, ConfirmDialog…
│   ├── lib/                       # Lógica de negocio y utilidades (ver §13)
│   │   ├── pdf/payroll-template.tsx    # Plantilla PDF
│   │   ├── excel/payroll-template.ts   # Plantilla Excel
│   │   └── __tests__/             # Tests Vitest
│   ├── types/                     # Tipos compartidos + next-auth.d.ts
│   ├── generated/prisma/          # Cliente Prisma generado (no editar)
│   └── proxy.ts                   # Middleware de Next.js 16
├── scripts/                       # Utilidades de mantenimiento (backups, fixes, e2e)
│   ├── capture-docs-screenshots.mjs   # Regenera las capturas del manual (Playwright)
│   └── build-docs-pdf.mjs             # Regenera los PDF desde los .md (Playwright, sin pandoc)
├── .github/workflows/             # backup-db.yml, migrate-db.yml
├── docs/
│   ├── img/                       # Capturas del MANUAL_DE_USUARIO (generadas, no editar a mano)
│   ├── INTEGRACION_INVENTARIO.md
│   └── superpowers/               # Planes y specs históricos
├── next.config.ts                 # Cabeceras de seguridad (CSP, HSTS…)
├── prisma.config.ts  vercel.json  vitest.config.ts
└── README / CLAUDE / DESPLIEGUES / BACKUP / DOCUMENTACION_COMPLETA / MANUAL_DE_USUARIO
```

7. Tecnologías y su propósito

Tecnología	Propósito
Next.js 16 (App Router)	Framework full-stack: Server Components, API routes, middleware.
TypeScript (strict)	Tipado estricto — cero `any`; errores de tipo en nómina = salarios erróneos.
Tailwind CSS v4 + shadcn/ui	Estilos utilitarios y primitivas de UI accesibles.
Prisma 7 ORM	Acceso a datos tipado; motor "client" con adapter.
libSQL / Turso	Base de datos SQLite distribuida (una por cliente en prod).
NextAuth v5 (Credentials)	Autenticación usuario/contraseña con sesión JWT.
bcryptjs	Hash de contraseñas.
@react-pdf/renderer	Generación de reportes de nómina en PDF.
exceljs	Generación de reportes en Excel.
date-holidays	Detección de festivos colombianos (para tarifa especial).
cloudinary	Almacenamiento de logos por tenant.
web-push	Notificaciones push (VAPID) al portal del empleado.
zod	Validación de todas las entradas de usuario antes de tocar Prisma.
react-hook-form + @hookform/resolvers	Formularios controlados con validación Zod.
Vitest	Tests unitarios (lógica de nómina, propinas, permisos, auditoría…).
Vercel	Hosting, cron jobs, variables de entorno por proyecto.

8. Dependencias principales

Ver versiones exactas en `package.json`. Producción: `next`, `react` / `react-dom` 19, `@prisma/client` + `@prisma/adapter-libsql` + `@libsql/client`, `next-auth` v5 beta, `bcryptjs`, `@react-pdf/renderer`, `exceljs`, `date-holidays`, `cloudinary`, `web-push`, `zod`, `react-hook-form`, `lucide-react`, `sonner`, `next-themes`, `class-variance-authority`, `clsx`, `tailwind-merge`. Desarrollo: `prisma`, `typescript`, `tailwindcss`, `vitest`, `@vitejs/plugin-react`, `tsx`, `eslint`, `@playwright/test`.

9. Configuración del entorno

Plantilla en `.env.example`. Variables (ver también la matriz por cliente en DESPLIEGUES.md):

Variable	Ámbito	Descripción
`TURSO_DATABASE_URL`	local + prod	Local: `file:./dev.db`. Prod: `libsql://…turso.io`.
`TURSO_AUTH_TOKEN`	prod	Token de la BD Turso del cliente.
`NEXTAUTH_SECRET`	local + prod	Secreto JWT (≥32 chars: `openssl rand -base64 32`).
`NEXTAUTH_URL`	local + prod	`http://localhost:3000` o la URL del despliegue.
`CLOUDINARY_CLOUD_NAME` / `_API_KEY` / `_API_SECRET`	prod	Almacenamiento de logos.
`CRON_SECRET`	prod	Protege `/api/cron/audit-purge` (Vercel Cron envía `Bearer`).
`AUDIT_RETENTION_MONTHS`	opcional	Meses de retención de auditoría (def. 6).
`VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY` / `NEXT_PUBLIC_VAPID_PUBLIC_KEY`	prod	Web Push.
`NEXT_PUBLIC_INVENTARIO_APP_URL`	opcional	Feature flag del inventario (vacío/ausente = OFF).
`AUTH_COOKIE_DOMAIN`	opcional (prod)	Dominio de la cookie de sesión compartida con el inventario (p. ej. `.cucinadeifiori.com`). Sin definir = cookie host-only por defecto. Ver §5.3.
`BACKUP_GPG_PASSPHRASE`	GitHub Actions	Cifra los backups semanales (secret del repo, no de Vercel).
`DB_TARGETS`	GitHub Actions	JSON array `[{name,url,token}]` con la BD de cada cliente; fuente de verdad de `backup-db.yml`. Ver BACKUP.md.
`NOMINA_SYNC_SECRET`	prod (con inventario)	Secreto compartido del webhook `/api/inventory-permissions/sync`.
`NX_PASSWORD_<USUARIO>`	solo terminal	Clave de cada usuario que crean o reinician los scripts operativos; si falta, se genera al azar.
`AUDIT_BASE_URL` / `AUDIT_USER` / `AUDIT_PASS`	solo terminal	Destino y cuenta de `scripts/e2e-prod-audit.mjs`.
`NX_ALLOW_REMOTE_SEED`	solo terminal	`1` permite correr una semilla de prueba contra una base remota (solo demo).

`.env.example` lista todas estas variables desde el 2026-11-05.

10. Instalación

Requisitos: **Node.js 20+** y npm.

```
npm install
cp .env.example .env.local        # y editar los valores (mínimo TURSO_DATABASE_URL=file:./dev.db,
npx prisma generate               # genera el cliente en src/generated/prisma
npx prisma db push                # crea/actualiza el schema en ./dev.db
npx tsx prisma/seed-local.ts      # datos de demostración (o: npm run db:seed)
```

11. Ejecución local

```
npm run dev      # http://localhost:3000
```

Otros comandos: `npm run build` (incluye `prisma generate`), `npm run start`, `npm run lint`, `npm run test`, `npx prisma studio`.

12. Base de datos

12.1 Modelos (19)

Modelo	Descripción
Tenant	Restaurante. Marca (colores/logo). Raíz de todas las relaciones.
User	Cuenta de acceso (username único, `passwordHash`, `role`, `active`). Puede vincularse a un `Employee` (`employeeId`), tener un `customRoleId` y `inventoryAccess`.
Employee	Empleado pagable. `hourlyRateNormal`, `hourlyRateSpecial`, `tipPercent`, `payType` (`PAYROLL` / `SHIFT`).
TimeEntry	Registro de horas de un día. `checkIn/checkOut` + `checkIn2/checkOut2` (turno partido). `isSpecial` (domingo/festivo).
Schedule / **ScheduleShift**	Horario semanal publicable y sus turnos por empleado/día (con posible segundo bloque).
PayAdjustment	Ajuste puntual de pago por período: `type` `BONUS` / `DISCOUNT`, `amount`, `description`.
TipEntry / **TipDistribution**	Propina de un día (total, menaje, neto, período) y su reparto por empleado (horas, %, horas efectivas, monto).
Bonus / **BonusAssignment**	Bono configurable (STANDARD/PER_EMPLOYEE, ALL/PAYROLL/SHIFT/SPECIFIC, BIWEEKLY/MONTHLY) y asignaciones/valores por empleado.
Discount / **DiscountAssignment**	Descuento configurable (mismo modelo que Bonus) y sus asignaciones.
CustomRole	Rol personalizado por tenant; permisos en JSON (`string[]`).
UserPermission	Override de permiso por usuario (`granted` true=concede / false=deniega).
AuditLog	Bitácora inmutable (append-only) de acciones.
InventoryCategory	Espejo local de categorías raíz del inventario externo.
PushSubscription	Suscripción de notificaciones push por usuario/dispositivo.
Holiday	Festivo **personalizado** del cliente (municipal/decretado). Se guarda como `month` / `day` + `year` opcional: `year = null` → se repite cada año; `year = 2026` → solo ese año. `@@unique([tenantId, year, month, day])`.

12.2 Relaciones clave

- **Todo cuelga de** `Tenant` (aislamiento por `tenantId`; una sola fila `Tenant` por base de datos en producción). `onDelete: Cascade` desde `Tenant`.
- `User 1–0..1 Employee` (un usuario puede representar a un empleado pagable).
- `TipEntry 1–N TipDistribution`; `Bonus 1–N BonusAssignment`; `Discount 1–N DiscountAssignment`; `Schedule 1–N ScheduleShift`.
- `User N–0..1 CustomRole` (`onDelete: SetNull`).

12.3 Reglas de persistencia (destacadas)

- `User.username` es **único global**. `TimeEntry` indexado por `(tenantId, employeeId)` y `(tenantId, date)`. `TipEntry` es **única por** `(tenantId, date)` (una propina por día).
- `BonusAssignment` y `DiscountAssignment` son únicas por `(bonusId/discountId, employeeId)`.
- `CustomRole` único por `(tenantId, slug)`; `InventoryCategory` única por `(tenantId, slug)`.
- **Bonos y descuentos NO persisten aplicaciones:** su efecto se **calcula stateless por quincena** al generar el reporte (evita duplicados y no altera históricos).
- **`AuditLog` es append-only:** nunca se edita ni se borra desde la app (solo lo purga el cron por retención). Los campos `username` / `role` son *snapshots* al momento de la acción.

12.4 Migraciones

- Migraciones versionadas en `prisma/migrations/` (init, split shifts, username en lugar de email, módulo de propinas, auditoría, roles y permisos).
- **Local:** `npx prisma db push` (o `migrate dev` para crear una migración nueva).
- **Producción (Turso, por cliente): no** usar `prisma migrate deploy`, `run-migrations.mjs` ni el workflow `migrate-db.yml`: el historial de `prisma/migrations/` está desincronizado desde junio de 2026. Cada cambio de esquema tiene su script idempotente `prisma/turso-migrate-*.mjs` con rollback, que se corre a mano en el demo antes del merge y en cada cliente antes de promoverlo. Ver DESPLIEGUES.md y BACKUP.md.

13. Módulos principales

Lógica de negocio en `src/lib/` (todo tipado, sin `any`):

Archivo	Responsabilidad
`payroll.ts`	Cálculo de horas y nómina (`calculateHours` , `calculatePayroll`).
`payroll-report.ts`	Armado del reporte enriquecido por período con carga **en bloque** (sin N+1).
`holidays.ts`	Festivos **nacionales** colombianos (`isSpecialDay` , `isColombianHoliday` , `listNationalHolidays` , `dateFromString`).
`special-days.ts`	Versión **tenant-aware** de día especial: domingo ∪ festivo nacional ∪ `Holiday` del cliente. Incluye el **recálculo retroactivo** (`recalculateSpecialForDates` , `affectedDatesForHoliday`). `server-only` .
`shift-times.ts`	Reglas de turno compartidas cliente+servidor: cruce de medianoche (`classifyShift` , `buildShiftDateTimes` , `validateShiftWindow`), tope diario (`MAX_DAILY_HOURS`) y aviso de jornada (`buildOvertimeWarning`).
`feature-flags.ts`	Punto único de lectura de feature flags de despliegue (`isInventoryEnabled`).
`tips.ts`	Cálculo y reparto de propinas (`calculateTips` , `getPeriodForDate` , `MENAJE_PERCENT`).
`bonuses.ts` / `bonus-service.ts` / `bonus-validation.ts`	Modelo, resolución por quincena y validación de bonos.
`discounts.ts` / `discount-service.ts` / `discount-validation.ts`	Ídem para descuentos.
`permission-keys.ts`	Catálogo de permisos, roles base, grupos, claves dinámicas de inventario.
`get-permissions.ts`	Resolución de permisos **efectivos** y acceso al área admin.
`permissions.ts` / `require-permission.ts`	Helpers de autorización en páginas/API.
`auth.ts`	Configuración de NextAuth (login, JWT, eventos, permisos de inventario).
`audit.ts` / `audit-labels.ts` / `audit-retention.ts`	Registro, etiquetas y purga de auditoría.
`pdf/payroll-template.tsx` / `excel/payroll-template.ts`	Plantillas de reporte.
`inventory-sync.ts`	Espejo local de categorías del inventario.
`logo-loader.ts`	Carga de logos (Cloudinary).
`push.ts`	Envío de notificaciones web push (VAPID).
`color-contrast.ts`	Auto-contraste de texto sobre colores custom del tenant.
`utils.ts`	`formatCurrency` , `formatDate` , `formatHours` , `todayColombia` , `cn` …
`report-types.ts`	Tipo `PayrollWithExtras` (nómina + propinas + bonos + descuentos).

14. Funcionalidades, una por una

14.1 Autenticación y sesión

Login por **usuario/contraseña**. Sesión JWT con rol, tenant y datos del empleado. Usuarios desactivados (`active=false`) no pueden entrar. Login, logout e intentos fallidos se auditan. La sesión dura 12 h sin actividad, y desactivar a un usuario o cambiarle la contraseña, el usuario o el rol cierra sus sesiones abiertas (§5.3).

14.2 Dashboard admin

Pantalla de entrada al área de administración; muestra los módulos accesibles según los permisos efectivos del usuario.

14.3 Personal (empleados)

Alta/edición/activación de empleados pagables: nombre, documento, teléfono, tarifas (normal/especial), `tipPercent`, `payType` (`PAYROLL` / `SHIFT`). Gestión de credenciales del empleado (crear/actualizar su cuenta de portal).

14.4 Registro de horas

Alta/edición de `TimeEntry` por empleado y día, con hasta dos bloques (entrada/salida) para turnos partidos. **`isSpecial` lo calcula el servidor** (nunca el cliente) según domingo / festivo nacional / festivo personalizado del tenant (`isSpecialDayForTenant`).

Reglas de turno (`shift-times.ts`, aplicadas en cliente y servidor):

- **Cruce de medianoche:** si la salida es ≤ la entrada, el turno se cierra en el día siguiente, **como máximo a las 02:00** (`MAX_OVERNIGHT_END_MINUTES = 120`). El registro conserva su `date` = día de inicio, así que todas las horas se pagan a la tarifa de ese día. Una salida posterior a las 02:00 se rechaza (`OVERNIGHT_LIMIT_ERROR`).
- **Tope diario:** la suma de turnos del día no puede pasar de **15 h** (`MAX_DAILY_HOURS`); el API responde 400.
- **Aviso de jornada:** superar **8 h** (`DAILY_ALERT_HOURS`) no bloquea, pero `buildOvertimeWarning` obliga a una confirmación explícita en la UI, mostrando los turnos del día ordenados por hora de entrada.
- **Máximo 2 registros por empleado y día**, sin solapamiento entre ellos. Estas reglas se revalidan también en el `PUT` cuando el registro se mueve de empleado o de fecha.

La lista tiene **paginación server-side** (tamaños 10/15/20, por defecto 15, máx. 20) que preserva filtros, y hace `select` solo de las columnas necesarias.

14.4.1 Festivos personalizados

Módulo `/admin/holidays`: además del listado **solo lectura** de festivos nacionales (por año), permite registrar festivos **decretados localmente** que `date-holidays` no incluye. Al crear, editar o eliminar uno, el

servidor **recalcula** `isSpecial` **de los turnos ya registrados** en las fechas afectadas y vuelve a repartir las propinas de esos días — sin re-capturar nada. Un festivo recurrente (`year = null`) afecta todas las fechas con ese mes/día en cualquier año; uno puntual, solo esa fecha.

14.5 Horarios

Creación de horarios semanales con turnos por empleado/día; publicación/despublicación (`published`). Los horarios publicados son visibles para el empleado en su portal.

14.6 Propinas

Registro de la propina total de un día. El sistema descuenta el **menaje (10%)** y reparte el **neto** de forma proporcional a las **horas efectivas** (horas × `tipPercent`). Una propina por día (`@@unique(tenantId, date)`). Recalculable (`recalculate-tips.ts`).

14.7 Bonos y descuentos

Definición de bonos/descuentos reutilizables: valor fijo para todos (`STANDARD`) o por empleado (`PER_EMPLOYEE`); alcance `ALL` / `PAYROLL` / `SHIFT` / `SPECIFIC`; frecuencia `BIWEEKLY` o `MONTHLY` (con `monthlyMode` `FIRST` / `SECOND` / `SPLIT`). Se aplican **automáticamente por quincena** en el reporte de nómina, sin persistir aplicaciones.

14.8 Ajustes de pago

Ajustes puntuales por período (`PayAdjustment`): `BONUS` (suma) o `DISCOUNT` (resta), con descripción. A diferencia de bonos/descuentos, son ad-hoc para un período concreto.

14.9 Reportes de nómina

Genera el consolidado quincenal/mensual por empleado: horas normales/especiales, bruto, bonos, descuentos, ajustes, propinas (informativas) y **total final**. Exportable a **PDF** y **Excel**. Reporte de **turnos** aparte.

14.10 Roles y usuarios

Gestión de usuarios del portal, roles personalizados (`CustomRole`) y permisos individuales (`UserPermission`). Solo `PROPRIETARY` (o quien tenga los permisos) gestiona roles/permisos.

14.11 Auditoría

Bitácora consultable (solo `PROPRIETARY` por defecto vía `audit:view`) de acciones: quién, qué, cuándo, sobre qué entidad, con `before/after` en ediciones e IP/user-agent.

14.12 Configuración del tenant

Nombre, colores primario/secundario y logo (Cloudinary). El auto-contraste asegura texto legible sobre los colores elegidos.

14.13 Portal del empleado

"Mi Quincena" (su liquidación del período), "Mi Horario" (turnos publicados) y "Perfil" (cambio de contraseña). Notificaciones push opcionales.

14.14 "Mi cuenta" para ADMIN con empleado

Un `ADMIN` vinculado a un empleado ve además "Mi Quincena" y "Mi Horario" dentro del área admin.

15. Roles, permisos y restricciones

15.1 Roles base

Rol	Alcance
PROPRIETARY	Acceso total (bypass): **todos** los permisos, calculado dinámicamente. Único que ve la auditoría por defecto y gestiona roles/permisos de inventario.
SUPERADMIN	Gestión completa: personal, horas, horarios, propinas, nómina, bonos, descuentos, ajustes, **festivos**, configuración, inventario.
ADMIN	Operación diaria: horas, horarios (solo ver), propinas, nómina y reportes, inventario. No gestiona personal, bonos/descuentos ni festivos.
EMPLOYEE	Sin permisos admin por defecto (baseline vacío). Solo su portal, salvo que reciba un rol personalizado o permisos individuales.

Los conjuntos exactos están en `BASE_ROLE_PERMISSIONS` (`src/lib/permission-keys.ts`).

15.2 Resolución de permisos efectivos (`get-permissions.ts`)

Orden:

1. Si el usuario es **PROPRIETARY** → todos los permisos (bypass).
2. Si tiene un **CustomRole activo** → usa sus `permissions` (reemplaza al rol base).
3. Si no → usa los permisos del **rol base**.
4. Se aplican los **overrides** (`UserPermission`): `granted=true` agrega, `false` quita.

`hasAdminAreaAccess()` concede el área admin si el usuario tiene **al menos un permiso** que no sea `profile:edit`. Así, un `EMPLOYEE` con rol personalizado que otorga acciones admin puede entrar, y el sidebar/páginas filtran por permiso.

15.3 Catálogo de permisos

Claves con el formato `modulo:accion` en `PERMISSIONS`. Módulos: usuarios, personal, roles, registro de horas, horarios, propinas, nómina, ajustes de pago, bonos, descuentos, **festivos** (`holidays:view|create|edit|delete`), configuración, auditoría, inventario, perfil.

Permisos **de inventario** (Nómina Xpress es la fuente de verdad; los consume la app externa): `inventory:view`, `products:create|edit|delete`, `categories:manage`, `stock:count`, `stock:adjust`, **`movements:edit`** (corregir movimientos manuales ya registrados), `daily:reopen`, `reports:view`, `users:manage` y **`audit:view`**. Además, claves **dinámicas** por categoría `inventory:daily:<slug>:<view|open|close|edit|history>` (ver docs/INTEGRACION_INVENTARIO.md).

16. Flujos de trabajo principales

16.1 Quincena típica (admin)

1. Registrar los **festivos locales** del período, si los hay (recalcula lo ya cargado).
2. Registrar/editar horas de cada empleado durante la quincena.
3. Registrar propinas del período (si aplica).
4. Configurar bonos/descuentos (una vez) y ajustes puntuales.
5. Generar el **Reporte de Nómina** del período → revisar totales.
6. Exportar a **PDF/Excel** para pago/archivo.

16.2 Cálculo de nómina (servidor)

`calculatePayroll(employee, entries, adjustments)`:

- Suma horas normales y especiales (incluye segundo turno si existe).
- `grossPay = normalHours × rateNormal + specialHours × rateSpecial`.
- Aplica ajustes (`BONUS` suma, resto resta) → `netPay`.
- En el reporte enriquecido (`PayrollWithExtras`): `finalPay = netPay + bonos − descuentos` (nunca negativo; las propinas son **informativas**, no se suman al total pagable).

16.3 Reparto de propinas (`calculateTips`)

- `menaje = round(total × 10%)`; `neto = total − menaje`.
- `horasEfectivas(e) = horasTrabajadas(e) × tipPercent(e)/100`.
- `tarifaHora = neto / Σ horasEfectivas`; `monto(e) = round(horasEfectivas(e) × tarifaHora)`.

16.4 Login → permisos → portal

Credenciales → NextAuth valida (bcrypt) → resuelve permisos efectivos y de inventario → firma JWT → el middleware enruta a `/admin` o `/portal` según rol. En cada request se verifica la versión de sesión y se resuelven de nuevo los permisos efectivos: un cambio de permisos individuales aplica de inmediato y un cambio de rol cierra la sesión.

17. Reglas de negocio

1. **Aislar por `tenantId` siempre.** Toda query filtra por el `tenantId` de la sesión (cada cliente corre con su propia BD de un solo tenant; nunca se comparte BD).
2. **Tarifa especial = domingo O festivo nacional O festivo personalizado del tenant.** Si un día cae en varias categorías, cuenta como un solo día especial. Se determina con `isSpecialDayForTenant(tenantId, "YYYY-MM-DD")` : `getUTCDay()===0 || isColombianHoliday()` resuelto en memoria, y solo si ninguno aplica se consulta la tabla `Holiday` .
3. `isSpecial` **siempre en el servidor** (tiene `date-holidays` y timezone correcto). Las fechas `YYYY-MM-DD` se anclan a **mediodía UTC** para que Colombia (UTC-5) caiga siempre en el mismo día.
4. **Un turno puede cruzar la medianoche hasta las 02:00** (hora de Colombia). El registro conserva `date` = día de inicio, así que **todas** sus horas se pagan a la tarifa de ese día. Tope de **15 h/día** por empleado y **máximo 2 turnos diarios** sin solaparse; superar **8 h/día** solo exige confirmación explícita del admin.
5. **Los festivos personalizados recalculan hacia atrás.** Crear/editar/eliminar un `Holiday` reescribe `isSpecial` de los `TimeEntry` de las fechas afectadas y recalcula sus propinas. Es la única mutación que toca registros históricos.
6. **Quincenas:** día ≤ 15 → `[día 1, 15]` ; si no → `[día 16, fin de mes]` . Helpers compartidos en `utils.ts` : `getBiweeklyPeriodForDate` , `getCurrentBiweeklyPeriod` , `getPeriodBounds` y `lastDayOfMonth` (último día dinámico 28/29/30/31, con strings `YYYY-MM-DD` sin `toISOString` → tz-safe). Las propinas usan además `getPeriodForDate` de `tips.ts` .
7. **Menaje de propinas = 10%** del total (`MENAJE_PERCENT = 0.1`).
8. **Bonos/descuentos son stateless por quincena:** un bono mensual de 2ª quincena no aparece en la 1ª; `SPLIT` reparte el valor entre ambas quincenas del mes.
9. `finalPay` **nunca es negativo**; las propinas no se suman al total pagable.
10. **Moneda en COP** sin decimales, formato `es-CO` (`formatCurrency`). Los cálculos se redondean a peso entero.

18. Validaciones importantes

- **Zod en todas las entradas** de usuario antes de tocar Prisma (regla no negociable). Validaciones específicas de bonos/descuentos en `bonus-validation.ts` / `discount-validation.ts`.
- **Autorización** en cada API route (permiso efectivo), no comparación literal de roles.
- **Claves de permiso válidas:** `isValidPermissionKey` acepta el catálogo estático o claves dinámicas de inventario diario por categoría.
- **Contraseñas** hasheadas con `bcrypt` (coste 12).
- **Unicidad** aplicada en BD (username, propina por día, asignaciones por empleado…, festivo por `(tenantId, year, month, day)`).
- **Reglas de turno duplicadas a propósito** en cliente y servidor (`shift-times.ts` es isomorfo): el cliente da feedback inmediato, el servidor es la autoridad (`validateShiftWindow`, tope de 15 h, máx. 2 turnos, sin solapes).
- **Reasignación de empleado en el `PUT` de horas:** `employeeId` está en el schema Zod, se verifica que el empleado destino pertenezca al tenant y se revalidan las reglas del día destino.

19. Manejo de errores

- **Auditoría nunca rompe la operación:** `recordAudit` está envuelto en try/catch y solo registra en consola si falla (regla: la auditoría se mantiene consistente aun ante errores).
- **Festivos tolerante a fallos:** `isColombianHoliday` usa try/catch para no romper por casos borde de `date-holidays`.
- **Sync de inventario tolerante:** si no hay URL de inventario, se usa el espejo local sin llamadas externas; el webhook de creación de categoría es best-effort.
- **Cron protegido:** `/api/cron/audit-purge` responde 500 si falta `CRON_SECRET` y 401 si el `Bearer` no coincide (evita un endpoint de borrado abierto).
- **API routes** devuelven códigos y mensajes JSON coherentes (401/403/500 según caso).

20. Seguridad

- **Aislamiento de datos** por `tenantId` en cada query, reforzado por el despliegue independiente (una base de datos por cliente).
- **Cabeceras de seguridad** en `next.config.ts` para todas las respuestas: `Content-Security-Policy` (restringe orígenes; imágenes solo self/data/blob/Cloudinary), `Strict-Transport-Security`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy` (cámara/micrófono/geo desactivados).
- **Contraseñas** con bcrypt; **sesión JWT** firmada con `NEXTAUTH_SECRET`, de 12 h y revocable por `sessionVersion` (§5.3). La auditoría nunca guarda hashes.
- **Autorización por permiso efectivo** en páginas y API (no confiar en ocultar botones).
- **Secrets fuera del repo:** tokens de Turso y credenciales por variables de entorno. Backups **cifrados con GPG/AES-256** (`BACKUP_GPG_PASSPHRASE`).
- **Cron de borrado** protegido por `CRON_SECRET`.
- **Scripts sin contraseñas:** los scripts operativos toman las claves de `NX_PASSWORD_<USUARIO>` o las generan al azar (`scripts/lib/credentials.mjs`); las semillas de prueba se bloquean contra bases remotas.
- **Repositorio:** secret scanning, push protection y Dependabot (alertas y PRs de seguridad) activos desde el 2026-11-05.

> **Incidentes de seguridad (históricos):**
>
> - Hubo exposición de tokens de escritura de Turso en el historial de git (rotados).
> - El 2026-11-05 se detectaron contraseñas de cuentas reales en scripts versionados de este repositorio público. Se retiraron del código y el propietario las cambió el mismo día; siguen en el historial de git, por eso cuenta solo el cambio de claves.
>
> Detalle en docs/RELEASE-fase0-seguridad.md.

21. Auditabilidad y trazabilidad

- `AuditLog` **append-only.** Cada evento guarda: `tenantId`, `userId` (nullable), `username` / `role` (snapshots), `action`, `module`, `entityId` / `entityLabel`, `description`, `before` / `after` (solo los campos que cambiaron, vía `diffChanges`), `ip`, `userAgent` (comprimido/omitir en eventos de auth), `result` (`SUCCESS` / `FAILURE`).
- **Acciones auditadas:** CREATE, UPDATE, DELETE, ACTIVATE, DEACTIVATE, PUBLISH, UNPUBLISH, EXPORT, PAYMENT, LOGIN, LOGOUT, LOGIN_FAILED. **Módulos:** AUTH, EMPLOYEES, TIME_ENTRIES, SCHEDULES, TIPS, PAYROLL, PAY_ADJUSTMENTS, BONUSES, DISCOUNTS, CREDENTIALS, SETTINGS, PROFILE, INVENTORY.
- **Helpers:** `recordAudit` (bajo nivel), `logAudit(req, session, entry)` (rellena tenant/usuario/IP), `getClientInfo` (extrae IP/user-agent).
- **Retención:** `purgeOldAuditLogs` borra lo más antiguo que `AUDIT_RETENTION_MONTHS` (def. 6). Ejecutado por **Vercel Cron** el día 1 de cada mes a las 04:00 (`vercel.json`: `0 4 1 * *`) contra `/api/cron/audit-purge`.
- **Visibilidad:** solo `PROPRIETARY` (permiso `audit:view`) por defecto.

22. Despliegue y producción

Ver DESPLIEGUES.md (fuente de verdad). Resumen:

- **Un solo código base (`main`).** El demo `nomina-xpress` auto-deploya `main`; cada cliente tiene una **rama puntero** `client/<cliente>` (fast-forward, sin código propio) como Production Branch en su proyecto de Vercel.
- **Diferencia entre clientes = solo Environment Variables** (feature flags + credenciales). BD **Turso propia** por cliente.
- **Runbook de promoción:** merge a `main` → verificar demo → `git merge --ff-only main` en `client/<cliente>` → push (auto-deploy) → verificar → rollback vía Instant Rollback si falla.
- **Alta de cliente:** crear BD Turso + `prisma db push`, seed, rama puntero, proyecto Vercel con sus env vars.
- **Backups per-cliente:** workflow semanal (domingos 03:00 UTC) que recorre el secret `DB_TARGETS`, vuelca cada BD, la cifra con **GPG/AES-256** y la archiva con retención de **180 días**. Si el backup falla, el workflow **abre un issue de GitHub** como alerta. Ver BACKUP.md.
- **Migraciones por cliente:** workflow `migrate-db.yml` sobre el mismo `DB_TARGETS`.
- **Cookie compartida con inventario:** si el cliente integra Inventory Xpress, definir `AUTH_COOKIE_DOMAIN` en **ambos** proyectos con el mismo valor y el mismo `NEXTAUTH_SECRET` (§5.3).

23. Mantenimiento

- **Regenerar cliente Prisma** tras cambiar el schema: `npx prisma generate`.
- **Aplicar cambios de schema:** local `prisma db push`; producción con el script `prisma/turso-migrate-*.mjs` del cambio, primero en el demo y luego en cada cliente, **antes** de desplegar el código (§12.4). **Siempre respaldar antes** (BACKUP.md).
- **Tests:** `npm run test` (Vitest) antes de mergear: 456 casos en 33 archivos al 2026-11-05. Cubren nómina, propinas, bonos, descuentos, concursos, permisos, sesiones y auditoría (`src/lib/__tests__/` y `__tests__/` de las rutas).
- **Lint/tipos:** `npm run lint` y el `build` (que corre `prisma generate` + `next build`).
- **Scripts de mantenimiento** en `scripts/` (backups, arreglo de índices/columnas, reset de usuarios, e2e). Revisar antes de ejecutar contra producción.
- **Auditoría:** verificar periódicamente que el cron de purga corre (logs de Vercel).
- **Documentación:** al añadir una funcionalidad, actualizar este documento, el MANUAL_DE_USUARIO.md y, si cambia arquitectura, `CLAUDE.md`.
- **Capturas del manual:** viven en `docs/img/` y son **generadas**, no se editan a mano. Tras un cambio de UI, con la BD sembrada (`npx tsx prisma/seed-local.ts`) y el servidor arriba (`npm run dev`), ejecutar:

  ```
  npm run docs:screenshots
  ```

  Sobrescribe los PNG con los mismos nombres que referencia el manual. Requiere el navegador de Playwright (`npx playwright install chromium` la primera vez).
- **PDF de la documentación:** `DOCUMENTACION_COMPLETA_DEL_PROYECTO.pdf` y `MANUAL_DE_USUARIO.pdf` son **generados** desde sus `.md` (fuente de verdad) con:

  ```
  npm run docs:pdf
  ```

  `scripts/build-docs-pdf.mjs` convierte Markdown → HTML → PDF con el Chromium de Playwright (sin pandoc/wkhtmltopdf). Regenerar ambos PDF cada vez que cambie alguno de los `.md` o las capturas de `docs/img/`, antes de entregarlos al cliente.

24. Recomendaciones para futuros desarrolladores

- **Respeta las reglas no negociables** de `CLAUDE.md` (aislar por `tenantId`, TypeScript strict sin `any`, `isSpecial` en servidor, Zod en entradas, Server Components por defecto, `formatCurrency`).
- **No hagas `fetch` desde Server Components** a API routes propias; usa Prisma directo.
- **Autoriza por permiso efectivo**, no por rol literal.
- **Añade un test** en `src/lib/__tests__/` para toda lógica de cálculo nueva.
- **Revoca sesiones donde corresponde:** toda ruta que desactive un usuario o cambie su contraseña, usuario o rol debe incluir `...sessionRevocationData()` en el `update` y tener su prueba (ver `src/app/api/admin/users/__tests__/session-revocation.test.ts`).
- **Feature flags nuevos:** `NEXT_PUBLIC_FEATURE_*` (cliente) / sin prefijo (server), leídos desde `src/lib/feature-flags.ts`.
- **Nunca commitees secretos.** Usa `.env*` (ya ignorados) y los secrets de Vercel/GitHub.

25. Limitaciones actuales

- **Inventario desactivado** por defecto (flag) y con la pantalla de matriz de permisos por categoría **pendiente/por validar** (ver docs/INTEGRACION_INVENTARIO.md).
- **Login sin límite de intentos** ni bloqueo progresivo, y contraseña mínima de 6 caracteres (Fase 1 del Plan de Calidad).
- **La CSP bloquea Google Fonts** (`fonts.googleapis.com`): la app se ve con la fuente de respaldo. Se corrige con la CSP con nonce o sirviendo las fuentes con `next/font` .
- **Inventory Xpress no verifica `sessionVersion` :** valida el JWT por su cuenta, así que una sesión revocada sigue viva allá hasta que la cookie expira (máx. 12 h).
- **Cálculo de nómina simple:** horas × tarifa + tarifa especial; **no** calcula recargos legales colombianos (nocturno, extras, prestaciones, seguridad social). En particular, las horas de madrugada de un turno que cruza medianoche se pagan a la tarifa del día de inicio, **sin recargo nocturno**.
- **Los festivos personalizados son por mes/día**, no por fecha móvil: un festivo que cambia de fecha cada año hay que registrarlo año por año (vigencia "solo un año").
- **Un cambio de tarifa no recalcula quincenas ya exportadas** (el reporte se recalcula al volver a generarlo, con las tarifas actuales).
- **Credenciales de seed heterogéneas** entre `seed.ts` , `seed-local.ts` y el demo en la nube; conviene alinearlas o documentar cuál aplica a cada entorno.
- **Migraciones a Turso** son scripts manuales por base, no CD automático.

26. Posibles mejoras futuras

- Unificar los seeds y las credenciales demo.
- Automatizar la promoción a clientes (CD) y las migraciones a Turso.
- Completar la UI de matriz de permisos de inventario por categoría y activarlo por cliente cuando se requiera.
- Añadir recargos legales configurables (nocturno/extras) si el negocio lo pide.
- Cobertura de tests de integración/E2E (ya hay `@playwright/test` y scripts e2e).
- Exportaciones adicionales y paginación/consulta avanzada en auditoría.